

PYXIS

Building a Mac Security Toolkit That Runs Locally

1. Introduction

Most people who care about the security of their own computer end up relying on someone else's software to tell them whether it is safe. Antivirus suites, cloud based dashboards, and subscription security apps all follow the same pattern. A piece of software phones home, a remote service interprets the results, and the user is handed a conclusion they cannot verify. The user is, in effect, renting their sense of security from a vendor rather than owning the tools that produce it.

Pyxis is a direct response to that pattern. It is an investigation toolkit for macOS that runs locally, built with a React frontend and a FastAPI backend, and it runs real system commands on the machine it is installed on, showing the user exactly what those commands returned. Nothing is uploaded to a third party, no telemetry is collected, and no verdict is silently attached to the results. This report covers what Pyxis is, how it is built, how it protects the user through its own security practices, and how it connects to the earlier CyberDefense project.

2. Motivation and Background

The starting point for this project was dissatisfaction with the idea of renting security. Commercial security products are often opaque about what they check, how often they check it, and what they do with the data they collect. Even well meaning consumer security suites tend to compress a large amount of system information into a single traffic light verdict without showing the underlying evidence. For a technically capable user, that abstraction is frustrating, since it hides the information needed to judge a finding independently.

Building your own tools costs time up front and will never match the polish of a commercial vendor. In exchange, it buys transparency. Every check that Pyxis runs is a real, inspectable command, and every line shown to the user is exactly what that command produced, with no hidden interpretation layer in between. This same philosophy carried forward from the author's earlier CyberDefense project, described later in this report.

3. Project Overview and Architecture

Pyxis describes itself as a toolkit for investigating a Mac locally, combining a React frontend, a FastAPI backend, and real Python underneath. First, it is local. Every check runs on the user's own machine, and nothing is transmitted elsewhere unless the user opts into the optional AI features covered later in this report. Second, it is an investigation toolkit rather than a passive monitor, since the user chooses which check to run and when. Third, it uses mainstream engineering tools rather than a bespoke stack, which keeps the codebase approachable and easy to extend.

The application presents two scrollable pages once its loading screen finishes. The first is a dashboard populated with live facts pulled from the backend while the loading animation plays, showing who the user is signed in as, the current time, the Mac's name and macOS version, and a live network latency check. The second page lists every available tool, and text typed into the dashboard's search box filters that list in real time. Opening a tool replaces both pages with a dedicated workspace, and a back button returns the user to the dashboard.

Architecturally, the project is organized as two cooperating halves started together by a single shell script. The frontend is a React application built on Vite, and the backend is a FastAPI service that wires together one router per tool. On the frontend, the tool catalog lives in a single source file, `src/lib/tools.js`, and a second file, `src/lib/api.js`, centralizes the one request path every tool workspace shares. On the backend, `main.py` stays intentionally thin, loading environment variables, restricting CORS to the local development server, and including one router per tool module. Each tool follows one consistent contract, a POST route that returns an ordered list of strings representing the exact terminal transcript the frontend should display, with no summarization or verdict folded in.

4. The Investigation Toolkit

Pyxis ships with ten tools grouped into five workspace stages that mirror how a person actually investigates a machine, covering hardware and storage, access, running programs, network activity, and file search. Every tool's Run button calls a real backend endpoint that shells out to standard macOS and Unix utilities, including ps, lsof, dscl, diskutil, find, ping, curl, dig, system_profiler, and Apple's networkQuality test, and returns their genuine output rather than a synthetic summary.

Tool	What it checks
See what's really going on inside my Mac	Hardware, OS version, storage, uptime, FileVault, firewall, and SIP status
Check my disk and storage health	Volume and disk information, plus an optional read-only diskutil verifyVolume check
See who's logged into my Mac	Local accounts, active sessions, recent logins, and failed sign-ins
What's running on my Mac right now?	Top processes by CPU usage and flags anything running from Downloads or temporary folders
What starts automatically when I turn on my Mac?	Login items, LaunchAgents, LaunchDaemons, and missing executables
What's my Mac connected to right now?	Established connections, listening ports, and UDP activity using lsof
Is my internet actually working?	Local network stack, router, public internet, DNS, and HTTPS tested separately
How good is my Wi-Fi signal?	Signal strength, noise, SNR, channel, PHY mode, and transmit rate
How fast is my internet, really?	Apple's native networkQuality test covering download, upload, and responsiveness
Find files or folders by name	Real find search across a selected folder, filtered by files, folders, or both

5. Security Engineering Practices

Because Pyxis is itself a security tool, its own implementation is held to a higher standard, and several concrete practices run through the codebase rather than being left as aspirations.

Typed input, rejected early.

Every backend endpoint that accepts user input defines a strong model for the request body. Malformed requests are rejected before any system command runs, closing off an entire class of bugs by construction rather than by defensive coding inside each handler.

No shell strings, ever.

Anywhere user input reaches a real system command, for example a hostname or a search term, that value is passed as a discrete list argument to `subprocess.run` rather than built into a shell string. Because no shell is invoked to parse the command, shell metacharacters in user input cannot be interpreted as anything other than literal text, which rules out command injection by design.

A narrow network surface.

CORS on the backend is restricted to the local development server's own origins. The backend is not meant to be exposed beyond the machine it runs on, and its configuration reflects that narrow scope.

Read only by default.

With one deliberate exception, every tool in Pyxis only reads information and does not change anything about the system it inspects. The one partial exception invokes Apple's own `diskutil verifyVolume` utility, which checks the file system for consistency without repairing or modifying it. Keeping the toolkit free from destructive changes means a user can run every check freely, without worrying that an investigation tool might itself become a source of harm.

6. The AI Layer and Scoped Assistance

Two optional features in Pyxis are backed by a real large language model call, and both are restricted to a single narrow purpose rather than acting as a general assistant embedded in the app. The dashboard search bar lets a user describe a problem in plain language and matches it to one real tool from the catalog, never an invented one. An Explain button appears once a tool has produced output, sending that exact terminal transcript to the model and returning a short, plain English summary ending with an explicit takeaway.

Both features are implemented using Groq's hosted inference API, gated behind a key the user supplies themselves. Without a key, both endpoints return a clear not set up response, and every other tool continues to work exactly as before, since the AI layer is additive rather than something the application depends on. Scoping is enforced twice. The system prompt sent to the model names exactly three permitted outcomes, reply to a greeting, match a genuine request to one tool from the supplied catalog, or decline as out of scope, and it treats any terminal transcript given to it purely as data to summarize, never as instructions to follow.

The more important layer is enforced entirely in application code and does not trust the model's output at all. When the suggestion endpoint receives a tool identifier back from the model, the backend checks that value against the real set of tool identifiers supplied in the same request before ever returning it to the frontend. Any identifier that is not a genuine, existing tool is discarded. This means that even if the model were induced to hallucinate a tool that does not exist, the application itself cannot act on that hallucination, since the suggestion never reaches the user. Output from a language model that will influence an application's behaviour should be validated in ordinary code, not trusted on the strength of a prompt alone.

7. Broader Toolkit Context

Pyxis did not appear in isolation. It follows an earlier project, referred to in my writing as CyberDefense, which served as an initial vehicle for the same philosophy, with a documented v0.1.0 release marking a first working milestone. A related piece describes integrating the VirusTotal public API into that project, extending it with the ability to check a file's reputation against dozens of antivirus engines at once, a capability that mirrors what Pyxis's own processes and startup tools do locally, for example flagging anything running from a Downloads or temporary folder, but reaching further to a shared threat intelligence source.

Pyxis is the more recent and more architecturally mature of the two, and its AI layer shows the same instinct carried into a new area. Even when an outside service is brought in, it is wrapped tightly enough that the application's core guarantees do not depend on trusting it blindly.

8. Workflow and Extensibility

Using Pyxis follows a short, repeatable loop. The user runs `./start.sh` from a terminal with a Python environment capable of running FastAPI already active, and the script starts both the frontend and backend together. The frontend becomes available at `http://localhost:5173`, and the backend exposes interactive documentation at `http://127.0.0.1:8000/docs`, letting a user test any tool's endpoint directly using FastAPI's generated interface.

From the dashboard, the user browses into the Tools page or types a plain language description of a problem into the search box. Opening a tool replaces the dashboard with its dedicated workspace, where any required inputs are collected through simple form fields. Pressing Run calls the tool's backend route and streams the resulting lines into a read only terminal, and pressing Explain sends that exact transcript to the scoped AI endpoint for a plain English summary.

Adding an eleventh tool is a mechanical process that carries little risk. On the backend, it means creating one file that defines a route returning the standard lines shape, then registering that module in `main.py`. On the frontend, it means adding one object to the tool catalog with a matching route. This consistent pattern is what keeps every tool in Pyxis, from disk verification to file search, built the same way.

9. Limitations and Future Directions

- Platform scope. Every tool shells out to utilities specific to macOS, so Pyxis in its current form only works on a Mac.
- Local machine scope. Pyxis investigates the single Mac it runs on and has no mechanism for surveying a fleet of machines.
- Optional AI features depend on a hosted API from an outside provider and a key supplied by the user, so anyone relying on them is, to that limited extent, trusting an outside service.
- The interpretive layer is intentionally thin by design, since tools report facts rather than verdicts, so a less technical user may still need the Explain feature to draw conclusions.
- A natural next step is deeper integration with the VirusTotal style reputation checking explored in CyberDefense, so flagged files could be checked against that source directly from within the app.

10. Conclusion

Pyxis shows that a genuinely useful security tool does not have to come from a vendor. Its architecture is ordinary and approachable, precisely so that it is easy to read, extend, and trust. Its security posture is enforced structurally, through typed inputs, subprocess calls that use argument lists, a narrow network surface, and a read only default, rather than through checks scattered through the code. Its one optional AI layer is scoped twice, once through careful prompting and once, more importantly, through an application level check that never lets unvalidated model output reach the user. Its connection to the author's earlier CyberDefense project shows a consistent conviction, that understanding and trusting your own tools is worth the effort of building them yourself.

References

Primary source material.

Eapen, B. "I Stopped Renting My Security. Then I Built Pyxis." <https://medium.com/@eapenbenny92/i-stopped-renting-my-security-then-i-built-pyxis-dade81165751>

Eapen, B. "Stop Renting Your Security. Build It." <https://medium.com/@eapenbenny92/stop-renting-your-security-build-it-707bb82023d3>

Eapen, B. "CyberDefense v0.1.0." <https://medium.com/@eapenbenny92/cyberdefense-v0-1-0-f1f1c64a6734>

Eapen, B. "Using the VirusTotal API with CyberDefense." <https://medium.com/@eapenbenny92/using-the-virustotal-api-with-cyberdefense-a3b7eece9213>

Eapen, B. "Python Just Got a Safer Large File Encryption. A Hands On Look at Cobblestone." <https://medium.com/@eapenbenny92/python-just-got-a-safer-large-file-encryption-a-hands-on-look-at-cobblestone-a119bf3c19e7>

Supporting technical resources.

FastAPI, official documentation <https://fastapi.tiangolo.com/>

VirusTotal, public API v3 documentation <https://docs.virustotal.com/>

VirusTotal, official Python client (vt py) <https://github.com/VirusTotal/vt-py>

Groq, API documentation for hosted LLM inference <https://console.groq.com/>